HA Trading Lab: Data Persistence and Recovery

Reviewed: 27 October 2026 | Scope: the simulated trading lab

Persistence means saving data so it survives a service restart. The lab combines durable storage, replication, transactions, acknowledgements, and duplicate detection. PostgreSQL is the financial source of truth; queues carry work between services.

```
Client -> HAProxy -> API -> PostgreSQL (order + reservation + outbox)
DB outbox -> Relay -> Kafka orders.submit -> Router -> Exchange
Exchange executions -> Router -> DB (fills + books + cursor + outbox)
DB outbox -> Relay -> Kafka orders.events -> Bridge
Bridge -> RabbitMQ notifications -> Worker -> DB notifications
```

1. API: save before acknowledging

The API writes the order, cash reservation, and two outbox messages in **one PostgreSQL transaction**. Either all commit or none do. It returns **202 Received only after commit**; this means accepted and saved, not filled.

If the response is lost after commit, the client retries with the same `client_ref`. Its unique database constraint prevents a second order. HAProxy and the API hold no essential order state that must survive their restart.

2. PostgreSQL: durable financial records

Orders, fills, balances, positions, outbox messages, and router progress live in PostgreSQL. Each database node stores its data in a named Docker volume, which survives container replacement unless explicitly deleted. PostgreSQL's write-ahead log (WAL) supports crash recovery.

Normally, quorum synchronous replication waits for the leader and one standby to durably record a commit. Patroni coordinates leadership through etcd; the poolers follow the new leader after failover. A returning replica replays missed WAL, using the archive when needed.

Configured exception: `synchronous_mode_strict=false` allows continued writes if no synchronous standby remains. During that condition, an acknowledged write may have only one database copy.

3. Outbox relay: retain work until Kafka confirms

The outbox is a durable to-do list stored with the order. Two relays select batches using `FOR UPDATE SKIP LOCKED`, publish to Kafka, and mark rows published only after all messages in the batch are confirmed.

If Kafka is unavailable, unpublished rows remain for retry. If a relay crashes after publication but before recording success, messages can be published again. Downstream processing must therefore tolerate repeats.

4. Kafka: persistent order queues

`orders.submit` and `orders.events` use disk-backed logs in Docker volumes, three replicas, `min.insync.replicas=2`, and producer `acks=all`. One broker can fail while the remaining in-sync majority continues processing. With insufficient replicas, publication waits or fails and the outbox retains unpublished work.

Router input offsets are saved in PostgreSQL with order claims. Separately, the notification bridges use Kafka's committed consumer offsets. Messages are retained for replay within the configured retention limits.

5. Router and exchange: recover uncertain outcomes

Two routers share a PostgreSQL lease; one is active. Epoch checks reject stale-router database transactions. The active router claims an order before sending it. If the exchange's response is lost, takeover/reconciliation queries the exchange by `client_ref` before deciding whether to resend.

Each fill's unique `exec_id`, account changes, position changes, order status, outbox event, and execution cursor are committed together. Replaying the same fill does not change the books twice. A database sweep also recovers delayed `PENDING` work independently of Kafka.

The simulated exchange is memory-only. Restarting it loses its own session state. The router detects a new exchange boot ID and cancels recorded resting orders from the old session. Exchange history is bounded, so arbitrarily long outages cannot be reconciled reliably. Market orders older than five seconds before submission expire; persistence does not guarantee execution.

6. RabbitMQ and workers: acknowledge after saving

The bridge publishes persistent messages (`delivery_mode=2`) to the durable `notifications` quorum queue, replicated across three RabbitMQ nodes with volume-backed storage. A majority must be available. It waits for RabbitMQ publisher confirmation before committing its Kafka offset.

A worker saves the notification in PostgreSQL before acknowledging RabbitMQ. A crash before acknowledgement allows redelivery; the unique `event_id` prevents duplicate storage. Rejected messages and messages exceeding the redelivery policy go to the dead-letter queue for review. In this lab, delivery means a saved notification, not an email or SMS.

7. Retention, backups, and boundaries

Data	Current policy
Orders and fills	Kept in PostgreSQL; no history deletion was added
Order/event Kafka topics	Six-hour retention plus a 50 MB-per-partition limit; size pressure can shorten replay history
Published outbox rows	Deleted when older than one hour; unpublished rows are retained
Saved notifications	Deleted after two hours
Price topics	One-hour retention plus size limits; separate A/B lines are single-broker topics
Database backups	Daily full, approximately 30-minute incrementals, continuous WAL archive; retain the newest full chain and required WAL

API price caches, browser connections, and dashboard session counters are memory state. APIs rebuild prices from incoming ticks; clients reconnect WebSockets. Redis supplies optional extras and is not the financial source of truth.

Persistence, replication, and backups solve different problems: restart recovery, surviving a node failure, and restoring historical state. All lab zones and the backup repository share one PC, so they do not protect against losing that PC or disk. A WSL shutdown stops availability even though intact volumes retain data.

Delivery is **at least once**, with keys and transactions preventing repeated database effects. This is not an unconditional exactly-once exchange-execution guarantee. Queue retention is finite, and the exchange has no durable state or enforced exchange-side idempotency.

Implementation references: `app/api.py`, `schema.sql`, `relay.py`, `router.py`, `bridge.py`, `worker.py`, `exchange.py`; `docker-compose.yml`; `images/postgres/patroni.yml` and `pgbackrest.conf`. These describe the implemented lab, rather than a production disaster-recovery guarantee.